PPL-Assignment 2 Answers for theoretical questions

ID1: 314833799 (Itay Kirgner)

ID2: 212438378 (Eran Shapira)

Question 1.1

Since `define` just gives a name to an expression, and that name can be used from that point on to reference the expression, every program in L1 can be transformed to L11 by just replacing the name that was defined as the expression itself (do that to all variables in the program).

Question 1.2

L2 added `lambda` and `if` statement, if we remove the special form `define`, it only "removes" the option to define lambdas in the program, which means writing procedures is not allowed. So we can replace every call to a procedure with writing the lambda code again. This will convert the program into L22.

Question 1.3

No, by limiting `lambda` this way, we can just rewrite every lambda with N parameters to be N lambdas with one parameter, they will call each other and the result will be the same.

Like (lambda (x) (lambda (y) (<something with x and y>))).

Question 1.4

Instead of accepting some function f, we will write the program with f already implemented in the function, this will be done for all the functions that were passed as an argument. So we can always transform our program to a program that does not accept functions with functions as parameters.

Question 1.5

They are required because they have different evaluation rules than primitive operators, for example in `if` statements: the interpreter will first calculate the first condition, and then depending on the result of it, we calculate the `then` or `else` part, only the one that is relevant, ignoring the other one. So if the part that is ignored, let's say `else`, had code that would lead to an error or infinite loop, with primitive operators the interpreter would have calculated it and failed or got stuck in a loop.

Question 1.6

No, it is not required, since in pure functional programming there are no side effects, an expression just returns a value, you can put 100 expressions one after the other but only the last one will be the returned value, and each expression would not affect the next expression. It is useful in non-pure functional programming, as they allow side-effects, like in imperative programming or procedural programming that we learned about in the beginning of the course. They will run a sequence of commands, each command can have side effects on the state of the program/machine/application.

Question 1.7

Lexical address is like a pointer to show exactly where a variable is defined in the program. Let's say we defined a `lambda` which takes some parameter `x`, and the body is another procedure that gets a parameter `x`, and then its body is some calculation like `(+ x y)`, and it runs with the value `(+ x 5)`.

Now the question is, "in `(+ x y)`, where are `x` defined and `y` defined?", lexical address would find its way to the `x` given as a parameter to the 2nd and `y`, since not defined in the lambdas, would be a global variable, while the lexical address of `x` that is `(+ x 5)` will find its way to the `x` given as the parameter to **first** `lambda`.

Question 1.8

Representing primitive operations as Closures makes it much easier to add a primitive to the interpreter as we only need to add the proper binding in the initialization of the global environment, the rest of the code is the same.

As for PrimOp, when the interpreter sees a PrimOp it can quickly identify which PrimOp it is and execute the needed primitive function that was defined for it – no need to look for anything else or perform any more steps, meaning it is more efficient and quicker than the Closure approach. If for example the program has a lot of PrimOp operations, and the program is executed by many users (applications like Facebook or a video game) then removing those extra steps to make it slightly faster becomes a huge deal in large scale.

Question 1.9.a

If a function in the program has a an expression, that we know it would be a costly operation to calculate (like a complicated algorithm for example), then we would want to calculate it only if we really have to.

A prime example would be an `if` statement:

Let's say a function gets `x` and `y` as input and someone is running the function with `x = some_complicated_function(10)` and `y = some_complicated_function(100)` and the function has an `if` statement that checks some condition, the `then` part simply returns `x` and `else` returns `y`, then there is no need to calculate both `x` and `y` in advance, better to just place the expression "as is" in the body – the interpreter will then run `x` or `y` based on the expression, and then `som_complicated_function()` will run only once instead of twice.

Question 1.9.b

This is the exact opposite, let's say a function gets parameter `x` and it is used 100 times in the function.

Let's say someone runs this function with `x=some_complicated_function(100)`, it would be a shame to calculate this value again and again 100 times. Instead, we can calculate it once when running the function, and then each reference of `x` will be replaced with the value calculated by `some_complicated_function(100)`.

Question 1.10.a

The function is used in the substitution model when the interpreter processes a program with applicative order, because the interpreter first needs to calculate the parameters (which are expressions of some kind) given to the function, it then needs to take the values the expressions outputted, and "place" them in the body of the function (not just a find-replace, doing substitution like learned in class) by replacing each reference of the parameters in the body with the calculated values. But it can't just put values in the body, as they interpreter expects it be a CExp, so the function valueToLitExp turns the values to expressions that can be safely placed in the body.

Question 1.10.b

In the normal evaluation, the interpreter does not calculate the expression given to the procedure, using the substitution process it places the given expressions "as is", since they are already expressions and that fits perfectly in the AST that the interpreter runs on.

Question 1.10.c

The environment model does not work the same way as the substitution model, instead of "putting" the values in the expressions, there are frames in the environment. And since the interpreter does not need to "update" the AST, it does not need to use valueToListExp, since there is no need to make the values "compatible" with the AST which expect a CExp.

Question 1.11.a

The “workflow” and logic of the environment model already address the possibility of having two “different” variables that are both named `x` (for example) by using the frames. The frames create separation when evaluating, such that with each closure, a new frame is created based on the environment the closure was called at, and if it will look for the needed variables in the current frame, and if not found, they will “go up” to the previous frame, and so on, meaning the first occurrence of the variable, is the one that is relevant for the code.

Question 1.11.b

It is not required to rename variables, because the whole risk of assigning the wrong value to a given variable is not relevant anymore – very variable is already bound to a value in the expression. Bound variables are always “safe”, it is the free variables that are at risk of being assigned the wrong value, so if the expression does not contain free variables, the bound variables are safe, so there is no need to rename any variables.